Global Explanations: Feature Effects

Lecture 2 – Explainable AI

Kristian Boroz

opencampus.sh

Week 02

Agenda

1. Motivation
2. Permutation Feature Importance
3. PDP and ICE
4. ALE
5. Interactions and limitations
6. Summary

Two questions about a feature

How much does it matter?

Importance: Permutation Feature Importance (PFI)

How does it matter?

Effect: PDP, ICE, ALE

Global, model-agnostic, post-hoc

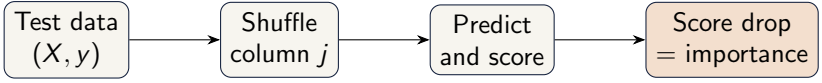
All four methods only call `model.predict`. They summarize the model's behaviour over the whole dataset, not for a single prediction (that is Lectures 3–4).

Running example: California housing, gradient-boosting regressor

Permutation Feature Importance (PFI)

Idea: if a feature matters, destroying its information should hurt the model.

1. Compute the baseline score s on held-out data
2. For each feature j: shuffle column j, recompute the score $s_{\text{perm},j}$
3. Importance: $\mathrm{PFI}_j = s - s_{\text{perm},j}$ (or the error ratio $e_{\text{perm}}/e_{\text{orig}}$)
4. Repeat with several shuffles; report mean and spread



PFI in scikit-learn

```
from sklearn.inspection import permutation_importance

pfi = permutation_importance(model, X_test, y_test, n_repeats=15, random_state=0)
imp = pd.Series(pfi.importances_mean, index=X.columns).sort_values()
imp.plot.barh(xerr=pfi.importances_std[imp.argsort()])
```

PFI: pitfalls

- Use held-out data: on training data, an overfit model can show importance for noise features (Breiman, 2001; Molnar, 2022)
- Correlated features: shuffling creates unrealistic rows (e.g. rooms $\gg$ bedrooms) and importance gets split between correlated features
- Randomness: always repeat and report the spread
- Importance $\neq$ direction: PFI says *how much*, not *how*
- Fix for correlated groups: permute groups of features together

Partial Dependence Plot (PDP)

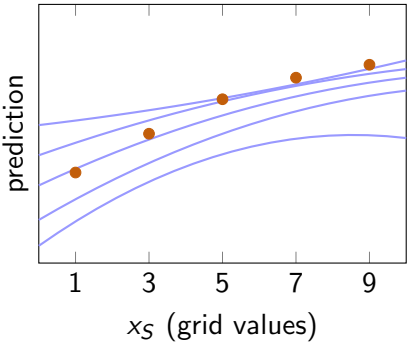
Set feature x_S to a value and average over the observed values of all other features x_C (Friedman, 2001):

$$\hat{f}_S(x_S) \;=\; \mathbb{E}_{X_C}\big[f(x_S, X_C)\big] \;\approx\; \frac{1}{n}\sum_{i=1}^{n} f\big(x_S,\, x_C^{(i)}\big)$$

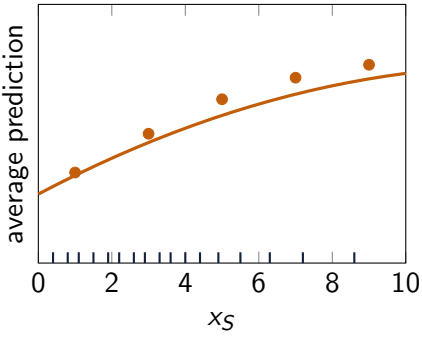
- Reads as: "average prediction if everyone had x_S"
- Needs a grid over x_S and n predictions per grid point
- 2D PDPs show the joint effect of two features

How a PDP is built

1. Predict for every instance



2. Average → PDP



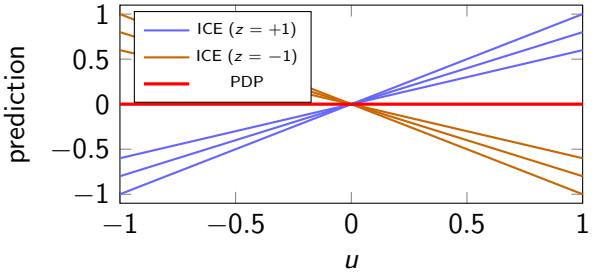
Schematic. Each blue curve is one instance with $x_C^{(i)}$ fixed; the PDP (orange) is their pointwise average at each grid value. The rug (bottom right) shows data density: where it is sparse, the PDP extrapolates.

PDP assumptions and limitations

- Independence: assumes x_S is independent of x_C. With correlated features it evaluates the model at unrealistic points
- Averaging hides heterogeneity: opposite effects can cancel out
- No data density: add a rug or histogram to see where the curve is supported
- Descriptive, not causal: it describes the model, not the world

ICE: one curve per instance

ICE (Goldstein et al., 2015) draws $f(x_S, x_C^{(i)})$ for every instance i. The PDP is the average of the ICE curves.



Schematic, $y = u \cdot z$, $z \in \{-1, +1\}$: flat PDP, yet u matters (notebook: random forest).

PDP and ICE in scikit-learn

```
from sklearn.inspection import PartialDependenceDisplay

PartialDependenceDisplay.from_estimator(
    model, X_train, ["MedInc", "HouseAge"], kind="both", # ICE + PDP
    subsample=60, random_state=0)

# 2D PDP
PartialDependenceDisplay.from_estimator(model, X_train, [("Latitude", "Longitude")])
```

Accumulated Local Effects (ALE)

Problem: PDP uses unrealistic feature combinations.
Idea (Apley & Zhu, 2020): only look at instances *inside a small interval* of x_j and accumulate local prediction differences.

$$\hat{f}_{j,\mathrm{ALE}}(x) = \sum_{k=1}^{k_j(x)} \frac{1}{n_k} \sum_{i:\, x_j^{(i)} \in N_k} \left[f\left(z_k, x_{\setminus j}^{(i)}\right) - f\left(z_{k-1}, x_{\setminus j}^{(i)}\right) \right] \; - \; c$$

- N_k: quantile bins of x_j, with edges z_{k-1}, z_k; c centres the curve
- Differences are taken with $x_{\setminus j}$ kept at its observed value, so no off-distribution points

PDP vs. ALE at a glance

	PDP	**ALE**
Question	Average prediction at x_j	Average local change around x_j
Correlated features	Can be misleading	Designed to handle them
Interactions	Averaged away	Main effect only
Interpretation	Absolute level	Relative to the mean prediction
Cost	$n\times$ grid size	$2n$ predictions
Ease of explanation	Very intuitive	Needs a short explanation

ALE from scratch (1D)

```
def ale_1d(model, X, feat, bins=10):
    edges = np.unique(np.quantile(X[feat], np.linspace(0, 1, bins + 1)))
    idx = np.clip(np.digitize(X[feat], edges[1:-1], right=True), 0, len(edges) - 2)
    eff, cnt = np.zeros(len(edges) - 1), np.zeros(len(edges) - 1)
    for k in range(len(edges) - 1):
        m = idx == k
        lo, hi = X[m].copy(), X[m].copy()
        lo[feat], hi[feat] = edges[k], edges[k + 1]
        eff[k], cnt[k] = (model.predict(hi) - model.predict(lo)).mean(), m.sum()
    ale = np.r_[0, np.cumsum(eff)]; mid = (ale[:-1] + ale[1:]) / 2
    return (edges[:-1] + edges[1:]) / 2, mid - np.sum(mid * cnt) / cnt.sum()
```

Libraries: `PyALE`, `alibi`. The notebook implements it by hand to expose the mechanism.

Feature interactions

- An interaction means the effect of x_j depends on another feature
- Detect: spread in ICE curves, 2D PDPs, or Friedman's H-statistic (share of variance from the interaction)
- Example: latitude and longitude jointly encode location: only the 2D plot shows the coastal price structure
- 1D plots show only main effects; high-order interactions are hard to visualize

Choosing a method

Question	**Start with**
Which features matter?	PFI on test data
How does x_j change predictions?	PDP + ICE, check the rug
Features correlated?	ALE, grouped PFI
Suspect interactions?	ICE, 2D PDP, H-statistic

Global plots can hide local behaviour: next lecture explains individual predictions with LIME and SHAP.

Summary and hands-on

Takeaways

- PFI: importance, on held-out data, repeated
- PDP/ICE: effect curves; ICE reveals heterogeneity
- ALE: robust to correlated features
- All describe the *model*, not causal truth

Notebook

1. PFI, train vs. test, grouped
2. PDP/ICE and 2D PDP
3. Flat PDP from an interaction
4. PDP vs. ALE (own implementation)

References

- Breiman (2001). Random forests. *Machine Learning*
- Friedman (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*
- Goldstein, Kapelner, Bleich, Pitkin (2015). Peeking inside the black box: ICE plots. *J. Computational and Graphical Statistics*
- Apley & Zhu (2020). Visualizing the effects of predictor variables in black box supervised learning models. *JRSS B*
- Fisher, Rudin, Dominici (2019). All models are wrong, but many are useful. *JMLR*
- Friedman & Popescu (2008). Predictive learning via rule ensembles. *Annals of Applied Statistics*
- Molnar. *Interpretable Machine Learning* (online book)